

MANUAL TÉCNICO Y DE ARQUITECTURA

Compilador Multi-Lenguaje Interoperable: Zetariano, Pig Latin y Y?

Curso: Compiladores 2

Autor: Elmer Miguel

Repositorio: https://github.com/ElmerMiguel/Compi2ProyectoSS2026_Zetariano

1. INTRODUCCIÓN Y OBJETIVOS

El proyecto consiste en un **compilador completo y autónomo de múltiples lenguajes de programación** desarrollado en **Java 21**, capaz de realizar análisis léxico, sintáctico, semántico, generación de código intermedio de tres direcciones (C3D), enlace de módulos independientes, traducción a lenguaje C estándar y generación de binarios ejecutables nativos mediante GCC.

El compilador resuelve la interoperabilidad en tiempo de compilación y ejecución de tres paradigmas y lenguajes distintos:

1. **Pig Latin (.pig)**: Lenguaje imperativo y punto de entrada principal del sistema (orquestador).
2. **Y? (.y)**: Lenguaje estructurado con sintaxis simplificada e indentación (similar a Python), soporte de estructuras anidadas (`struct`), arreglos y funciones.
3. **Zetariano (.z)**: Lenguaje orientado a objetos puro con soporte de clases, herencia, encapsulamiento, atributos, constructores y métodos polimórficos.

2. ARQUITECTURA GENERAL DEL SISTEMA

El compilador sigue una arquitectura basada en **tuberías y filtros (Pipes & Filters)** dividida en fases secuenciales que transforman el código fuente en binario ejecutable.

2.1. Diagrama de Arquitectura General

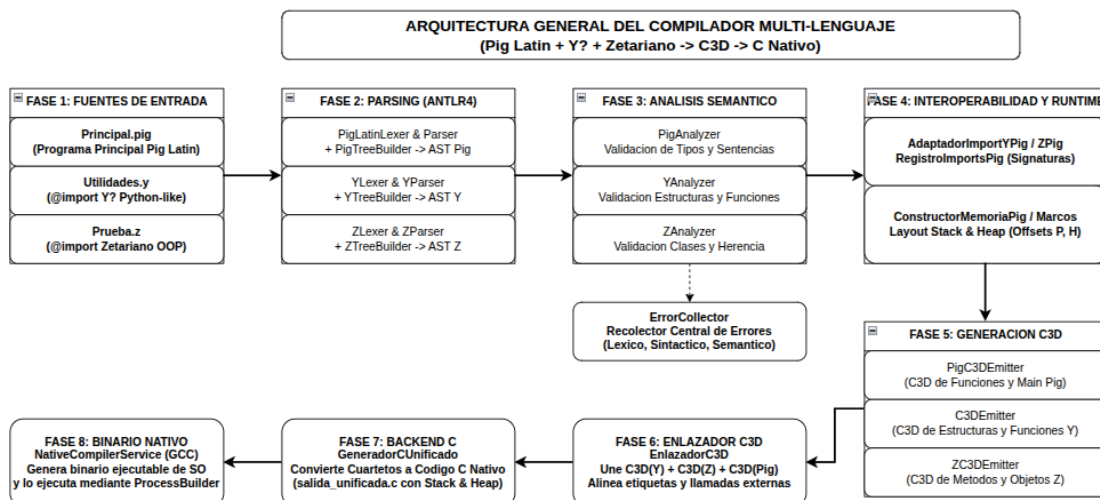


Figura 1: Arquitectura General del Pipeline de Compilación (Fuentes -> ANTLR -> AST -> Semántica -> C3D -> Enlace -> C Nativo -> Binario).

3. ESPECIFICACIÓN DE LOS LENGUAJES Y COMPONENTES

3.1. Pig Latin (Lenguaje Orquestador)

- **Punto de Entrada:** Posee la función `principal()` o bloque ejecutable donde inicia el programa.
- **Directiva `@import`** : Permite importar módulos externos `.y` y `.z`.
- **Sintaxis:** Inspirada en lenguajes procedurales clásicos. Admite variables primitivas (`entero` , `flotante` , `cadena` , `caracter` , `bool`), estructuras de control (`si` , `mientras` , `hacer mientras` , `para` , `elegir/caso`), lectura (`leer`), escritura (`escribir`), creación de arreglos y llamadas directas a funciones o métodos importados.

3.2. Y? (Python-Like)

- **Paradigma:** Procedural / Funcional estructurado.
- **Características:**
 - Declaración de estructuras (`estructura Nombre { ... }`).
 - Funciones con tipado estático y paso de parámetros por valor y por referencia.
 - Soporte de arreglos multidimensionales y estructuras anidadas.

3.3. Zetariano (Orientado a Objetos)

- **Paradigma:** Programación Orientada a Objetos.
- **Características:**
 - Clases con visibilidad pública y privada.
 - Atributos de instancia y métodos con paso de mensajes (`this`).
 - Constructores para instanciación dinámica en Heap (`nuevo Clase(...)`).
 - Herencia simple con propagación de miembros y despacho de métodos.

4. FASE DE ANÁLISIS LÉXICO Y SINTÁCTICO

La generación de analizadores léxicos y sintácticos se implementa mediante **ANTLR 4.13.2**:

Lenguaje	Lexer	Parser	Visitor / AST Builder
Pig Latin	<code>PigLatinLexer.g4</code>	<code>PigLatinParser.g4</code>	<code>PigTreeBuilder.java</code> -> <code>ProgramaASTPig</code>
Y?	<code>YLexer.g4</code>	<code>YParser.g4</code>	<code>YTreeBuilder.java</code> -> <code>NodoAST</code>
Zetariano	<code>ZLexer.g4</code>	<code>ZParser.g4</code>	<code>ZTreeBuilder.java</code> -> <code>ProgramaASTZ</code>

Los analizadores transforman el flujo de caracteres en tokens y construyen el CST (Concrete Syntax Tree).

Inmediatamente, la capa de constructores (`*TreeBuilder`) desacopla el compilador de ANTLR generando nodos AST independientes que facilitan el análisis semántico y la optimización.

5.1. Diagrama de Clases del Compilador y Semántica



- **PigAnalyzer.java** : Verifica el alcance de variables locales y globales, valida tipos en expresiones binarias/unarias, y consulta `RegistroImportsPig` para validar llamadas a librerías externas.
- **YAnalyzer.java** : Valida la coherencia de estructuras (`StructDef`), tipos de retornos de funciones, unicidad de nombres y pasaje de parámetros.
- **ZAnalyzer.java** : Valida la jerarquía de clases (`TablaClasesZ`), firmas de constructores y métodos, y resolución de accesos a miembros (`objeto.metodo()` o `objeto.atributo`).

- `enterScope()` / `entrarAmbito()` : Crea un nuevo nivel léxico para bloques (`if`, `while`, funciones).
- `exitScope()` / `salirAmbito()` : Cierra el entorno actual y libera la visibilidad local.
- `add(Symbol s)` : Registra un símbolo validando colisiones en el mismo ámbito.
- `lookup(String name)` : Búsqueda jerárquica desde el ámbito actual hacia los ámbitos padre/global.

- `ErrorKind` : Enumeración que clasifica el fallo (`LEXICO` , `SINTACTICO` , `SEMANTICO`).
- `ErrorReport` : Objeto inmutable que almacena tipo, mensaje exacto, línea y columna.
- `AntlrErrorBridge` : Conecta los oyentes de error de ANTLR al recolector global.

6. INTEROPERABILIDAD ENTRE LENGUAJES Y FLUJO DE COMPILACIÓN

6.1. Diagrama de Flujo e Interoperabilidad

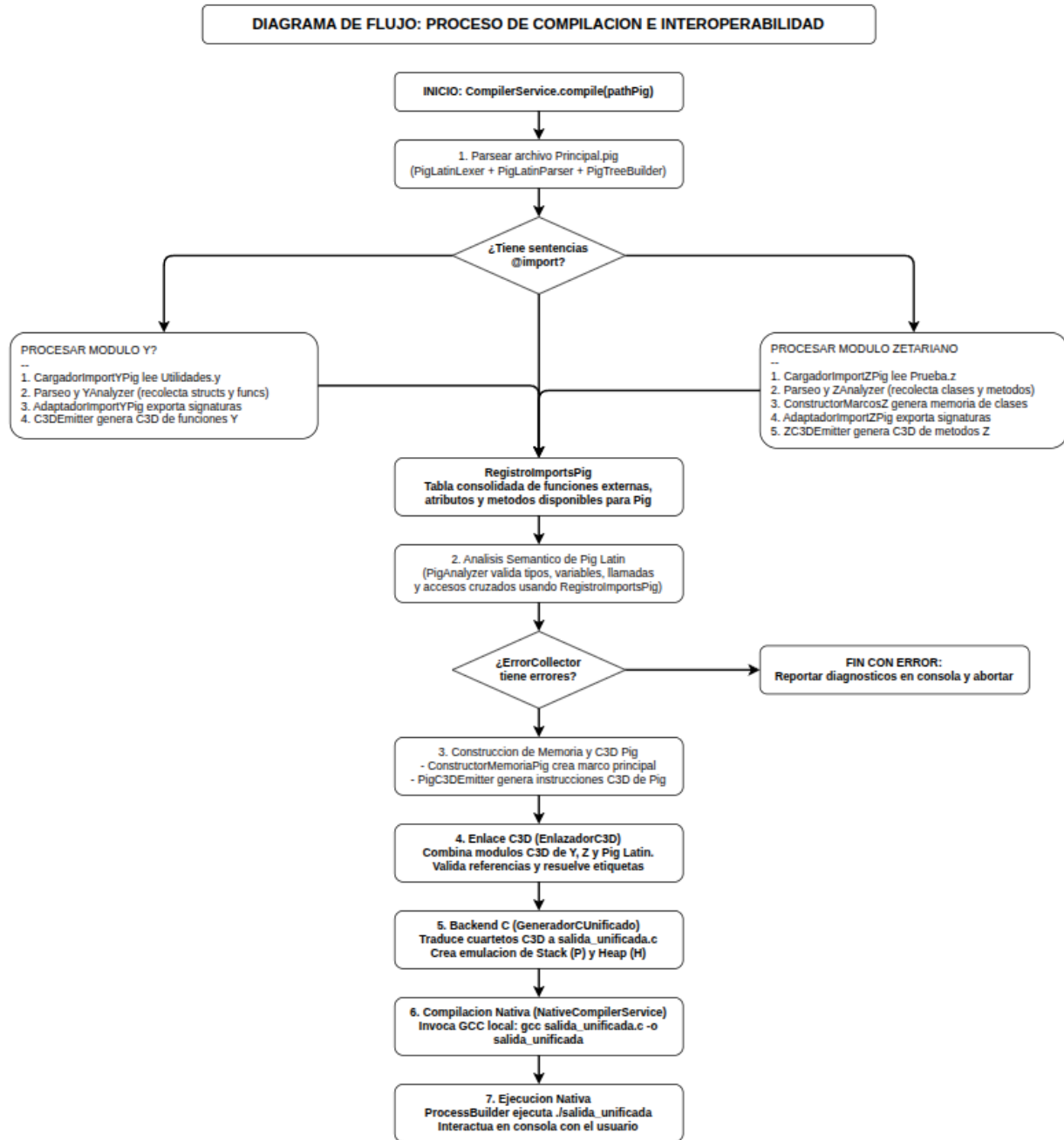


Figura 3: Diagrama de Flujo del Proceso de Compilación, Carga de Módulos @import y Enlace Multi-Lenguaje.

6.2. Mecanismo de Resolución de Imports

Cuando `Principal.pig` declara:

```
@import "Utilidades.y"
@import "Prueba.z"
```

1. `CargadorImportYPig` analiza `Utilidades.y` y produce un `YAnalyzer`. `AdaptadorImportYPig` extrae las funciones y structs y los inscribe en `RegistroImportsPig`.
2. `CargadorImportZPig` analiza `Prueba.z` y construye `TablaMemoriaZ`. `AdaptadorImportZPig` extrae las clases, constructores y métodos y los inscribe en `RegistroImportsPig`.
3. Al compilar `Principal.pig`, cualquier llamada como `sumar(12, 18)` o `nuevo Operaciones(5)` se valida contra este registro consolidado.

7. MODELO DE MEMORIA EN TIEMPO DE EJECUCIÓN (RUNTIME)

El entorno de ejecución emula una máquina virtual con dos segmentos de memoria lineal (`float` / `double`):

7.1. Diagrama del Modelo de Memoria y Marcos de Activación

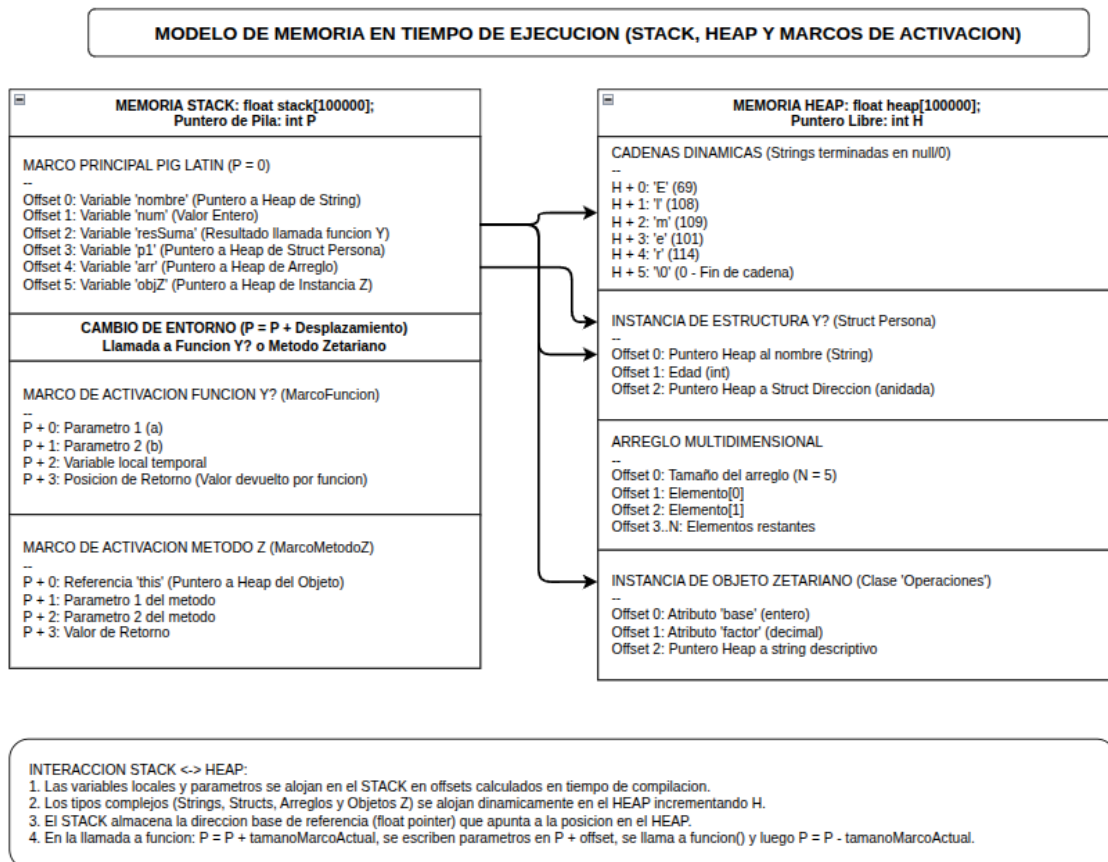


Figura 4: Estructura del Stack con Puntero P, Heap con Puntero H y Marcos de Activación.

7.2. Detalle de Segmentos

- **Puntero de Pila (P):** Apunta a la base del marco de activación actual. En llamadas a subrutinas:
 1. `P = P + tamaño_marco_actual;`
 2. Se colocan parámetros en `stack[(int)(P + offset_parametro)] = valor;`
 3. Se ejecuta `call funcion;`
 4. Se lee el retorno en `stack[(int)(P + offset_retorno)];`
 5. `P = P - tamaño_marco_actual;`
- **Puntero de Montículo (H):** Crece monótonamente al reservar memoria dinámica para strings, objetos, structs y arrays.

8. GENERACIÓN DE C3D, ENLACE Y BACKEND

8.1. Diagrama de Clases C3D y Backend

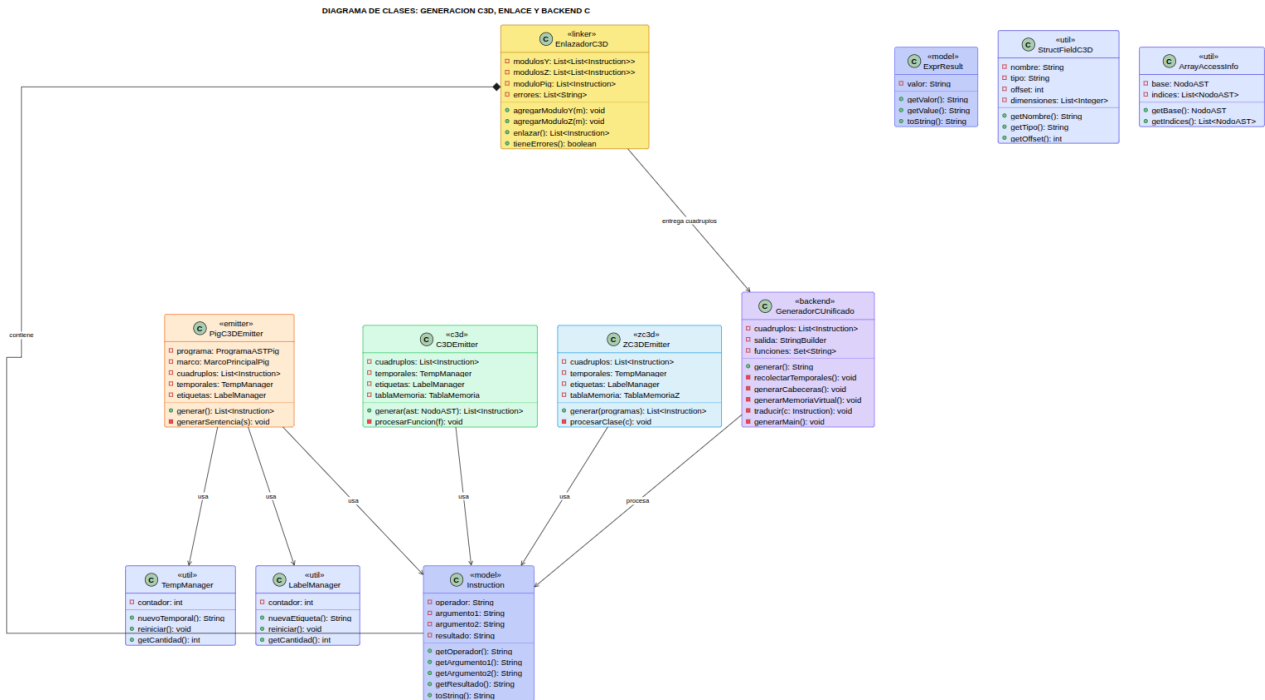


Figura 5: Diagrama de Clases UML del Subsistema de Código Intermedio C3D, Enlazador y Backend C.

8.2. Cuartetos (Instruction.java)

Cada instrucción C3D representa una tupla (operador, arg1, arg2, resultado) :

- Aritméticas: (+, t1, 5, t2)
- Saltos: (LABEL, -, -, L1), (GOTO, -, -, L2), (IF_EQ, t1, 0, L3)
- Memoria: (SET_STACK, P, t1, -), (GET_STACK, P, -, t2), (SET_HEAP, H, t3, -)
- Llamadas: (CALL, -, -, funcion)

8.3. Enlazador C3D (EnlazadorC3D.java)

Combina las listas de instrucciones generadas por C3DEmitter, ZC3DEmitter y PigC3DEmitter :

1. Normaliza las etiquetas para evitar colisiones entre módulos.
2. Asegura que la subrutina `main` de Pig Latin quede al inicio del flujo ejecutable.
3. Valida que todas las etiquetas destino y funciones referenciadas existan en el conjunto enlazado.

8.4. Backend C Unificado (GeneradorCUnificado.java)

Genera un archivo fuente C (`salida_unificada.c`) con:

- Inclusión de cabeceras estándar (`<stdio.h>`, `<stdlib.h>`, `<string.h>`).
- Declaración del Stack `float stack[100000];` y Heap `float heap[100000];`.
- Declaración de punteros `int P = 0;` `int H = 0;`.
- Funciones auxiliares de I/O (`imprimir_cadena`, `leer_cadena`, `concatenar_cadenas`).
- Emisión secuencial de instrucciones en funciones C equivalentes y la función `int main()`.

8.5. Compilador Nativo (NativeCompilerService.java)

Invoca el compilador del sistema operativo:

```
gcc -O2 salida_unificada.c -o salida_unificada
```

Produce un ejecutable binario listo para interactuar con la terminal o el IDE.

9. ÍNDICE DE RECURSOS Y DIAGRAMAS PARA DRAW.IO

A continuación se lista la correspondencia entre los archivos fuente de Draw.io (.txt) y las imágenes exportadas (.png):

#	Archivo Fuente Draw.io	Imagen Exportada	Descripción del Diagrama
1	diagrama_architectura_general.txt	imagenes/diagrama_architectura_general.png	Arquitectura de 8 fases del compilador
2	diagrama_clases_compilador.txt	imagenes/diagrama_clases_compilador.png	Clases UML de análisis y símbolos
3	diagrama_flujo_compilacion_interoperabilidad.txt	imagenes/diagrama_flujo_compilacion_interoperabilidad.png	Flujo de resolución de @import y pipeline
4	diagrama_modelo_memoria_runtime.txt	imagenes/diagrama_modelo_memoria_runtime.png	Layout de Stack, Heap y marcos de activación
5	diagrama_clases_c3d_backend.txt	imagenes/diagrama_clases_c3d_backend.png	Clases UML de C3D, enlazador y backend C

10. VALIDACIÓN Y PRUEBAS DEL COMPILADOR

10.1. Pruebas Unitarias Automatizadas

El compilador cuenta con una suite de pruebas con **JUnit 5**:

- `PigAnalyzerTest` : Validación de análisis semántico de Pig Latin.
- `YAnalyzerTest` : Validación de análisis semántico de Y? y estructuras anidadas.
- `ZAnalyzerTest` : Validación de clases y métodos en Zetariano.
- `C3DEmitterTest` : Generación de código intermedio y enlace modular.

- `UnifiedCGeneratorTest` : Generación completa de código C y verificación de enlace.

10.2. Prueba Integral Multi-Lenguaje (`docs/Pruebas/Principal.pig`)

La prueba de integración orquesta los 3 lenguajes simultáneamente:

1. **Entrada de usuario interactiva:** Lectura de nombre mediante `leer(nombre)` .
2. **Llamada a función de Y?:** `sumar(12, 18)` -> produce `30` .
3. **Estructura en Y?:** Creación de `Persona` con dirección anidada y lectura de campos.
4. **Modificación de arreglo por referencia en Y?:** Arreglo modificado de `10` a `999` .
5. **Instanciación y métodos en Zetariano:** Creación de `operaciones` , cálculo de factorial de 5 (`120`) y concatenación dinámica de cadenas en Heap.
6. **Resultado:** Código de salida `0` sin excepciones en tiempo de compilación ni ejecución.